Report Esercitazione 2

Gli indirizzi degli array e delle variabili x e y sono i seguenti, riportati sia in esadecimale che in decimale:

- **x**: 0x7ffe03711590 - 140728956163472
- **y**: 0x7ffe0371158c - 140728956163468
- **ad**: 0x7ffe037115c0 - 140728956163520
- **af**: 0x7ffe037115a0 - 140728956163488
- **ai**: 0x7ffe03711594 - 140728956163476

Gli oggetti sono memorizzati in fila: le distanze tra gli indirizzi degli oggetti corrispondono alle dimensioni dell'oggetto posizionato prima. Il primo è y, variabile di tipo float, di dimensione 4 byte. Il secondo è x, il cui indirizzo è di 4 unità più grande rispetto a quello di y. Dopo 4 byte (dimensione di una variabile di tipo int) c'è ai, array di interi, che occupa 4*3 (dimensione elemento * numero elementi) byte. Quindi, dopo 12 byte c'è af, array di float che occupa 4*8=32 byte. Infine vi è ad, array di double che occupa 4*8=32 byte.

Rappresentazione grafica della memoria:

L'istruzione alla riga 14 del programma, con l'aggiunta delle istruzioni per la stampa degli indirizzi di x e y, dà come output 0. Questo perché l'istruzione a riga 12 - (&y)[1] = 0; - prende l'indirizzo di y e accede all'elemento situato 4 byte più avanti, corrispondente alla dimensione di un float (ovvero tratta l'indirizzo di y come un array e si muove di un elemento in avanti); a questo punto l'elemento viene impostato a 0. Ma, come visto prima, a quell'indirizzo si trova x: il suo valore originale (1) viene sovrascritto.

Osservazione: Nel caso avessi compilato con gcc il programma così come in Figura 1, senza aggiungere la stampa degli indirizzi di x e y, il risultato sarebbe stato 1. Facendo debugging, ho notato che in quel caso gli indirizzi di x e y non erano vicini, mentre il "malcapitato" sarebbe stato ai. Invece compilando con clang avrei ottenuto comunque 0 come output. Questo dimostra che il posizionamento in memoria dipende strettamente dalle scelte di ottimizzazione dello specifico compilatore.